

AI Infrastructure Resource Ledger: The Reality Check

Thursday, 5. March 2026

Engineer(s): Antony Zhao

Date: 3/5/26

Project Title: Multi-Modal Latent Goal Alignment

Phase 1: The Calendar (Working Backward)

If you are still writing your training loop on your Drop-Dead Date, your project will fail.

1. Final Demo / Presentation Date: 4/27
2. Days required for Evaluation & Report: 7 days.
3. Estimated Training Run Duration: 3 days for LIBERO-100 midtraining, 1 day for final fine-tuning.
4. DROP-DEAD TRAINING START DATE: 4/17 (*Subtract Line 2 & 3 from Line 1*)

Phase 2: The Stack & Compute Target

1. Base architecture backbone, include encoders etc., add as many lines as you need: DINOv2 (300M, frozen), SigLIP (150M language encoder, frozen), Projection Heads (~10M between all), Policy Transformer (100M)
Param Count: 0.6 Billion / 110M trainable
2. Target Compute Hardware:
☒ Local / Lab Node
☐ Cloud Rental (e.g., Lambda/AWS)
3. Specific GPU(s): 3090
Total VRAM Available: 24 GB

Phase 3: The Memory Math (The VRAM Ledger)

Reference: 1B Parameters $\approx$ 2GB (Weights) + 2GB (Gradients) + 12GB (AdamW Optimizer States)

1. Model Weights (BF16): 1.2 GB
2. Gradients (BF16): 0.2 GB
3. Optimizer States: 1.2 GB
4. KV Cache & Activations Reserve: ~20 GB
5. TOTAL REQUIRED VRAM: 12 GB

The Strategy: Is your Total Required VRAM > Total Available VRAM?

☐ Yes ☒ No

If Yes, what is your exact mitigation strategy? ☐ FSDP / ZeRO-3 (Sharding across multiple GPUs)

☐ LoRA / QLoRA (Freezing base weights, avoiding optimizer state explosion)

☐ I am shrinking my model selection.

☐ Something else (explain):

Phase 4: The Data Ingress (The I/O Bottleneck)

1. Dataset Size: 23 GB / _____ Total Samples
<https://huggingface.co/datasets/lerobot/libero>
https://huggingface.co/datasets/jesbu1/bridge_v2_lerobot
2. Format:
☒ Raw JPEGs/PNGs/trajectories/etc. in a folder (High I/O Risk)
☐ WebDataset / Sharded .tar / MDS
☒ Something else: hybrid between sharded parquet files and videos that need to be clipped and sharded into images
3. Storage Location:
☒ Local NVMe SSD
☐ Network Drive
☐ S3/GCS Bucket
☐ Something else

If using raw images or other large data quantities over a network or standard SSD, how will you prevent your dataloader from starving your GPUs?

Phase 5: Red Team Audit (Partner Review)

Reviewing Engineer: _Heckman_____

Look at your partner's ledger. Find the fatal flaw. Are they underestimating training time? Do they have a 40GB model fitting into 24GB of VRAM? Are they trying to load 2 million JPEGs sequentially?

The Fatal Flaw / Biggest Risk:

Some concern about full fine tuning potentially not fitting within the 24GB budget; explore bitsandbytes for quantization along with gradient checkpointing. If activations memory usage remains high, move to A100 (concerns raised here about memory bandwidth from cloud storage). Combinations of datasets used for training (cross product) permutations of frozen/unfrozen/fine-tuning is unclear based on experimental plan; this could result in a (e.g.) 10x10 table from which we need to select e.g. 5 experiments. After

which, we would run an evaluation (inference) on those 5 models.

Phase 6: The 72-Hour Contract

What exact, granular engineering task will you complete in the next 72 hours to unblock these risks? (e.g., "Write dummy dataloader script to verify throughput," "Run a 10-step overfitting batch to check VRAM usage.")

Deliverable by Monday: Do a deep dive into the actual datasets to verify what they're like and how many samples there are. Also, write a small training loop to fully verify VRAM requirements.